

**超低位元參數高效率遷移式學習之 AI 硬體加速器
架構設計與實作**

**Hardware Architecture Design and Implementation of
Parameter-Efficient Transfer Learning for Ultra-Low-Bit AI
Accelerators**

Author: Po-Chun Huang (黃柏鈞)

Advisors: Pao-Ann Hsiung (熊博安)

Chun-Hsien Huang (黃駿賢)

July 14, 2026

Abstract

Edge devices are inherently resource-constrained [1, 31]. Published approaches for multi-task AI inference on edge FPGAs fall into three families, each with a documented cost: (a) partial-reconfiguration (PR) time-multiplexing with measured switching latencies of 97–114 ms via PCAP [20] and 5.2 ms via ZyCAP [18] and a hard requirement for a reconfiguration controller; (b) fixed feature extractor with programmable back-end (FixyNN [39]), whose front-end is frozen at tape-out and therefore cannot adapt at runtime; and (c) on-chip model/classifier selection (NeuroAdaptiveNet [12]), which keeps every variant always-resident on-chip. Within the cited works reviewed in this thesis, we did not identify a system that simultaneously offers sub-millisecond task switching *and* feature-extractor-level task adaptation on a 1W1A CNN backbone; the contribution of this thesis is therefore scoped against this explicitly cited set of prior work rather than claimed as universally novel.

In recent years, parameter-efficient fine-tuning (PEFT) has demonstrated strong potential across various AI domains, enabling adaptation to new tasks by updating only a small fraction of parameters [35, 40]. However, practical hardware implementation under ultra-low-bit quantization remains underexplored. In particular, when the CNN backbone is quantized to 1-bit weights and 1-bit activations (1W1A), the representational capacity is severely constrained [47, 50], and directly transplanting software-side adapter mechanisms into hardware introduces both accuracy degradation and integration challenges.

This thesis proposes **Barkaic**, a proposed AI hardware accelerator architecture that addresses both the accuracy and deployability bottlenecks of ultra-low-bit adapter-based inference. The proposed approach combines three key techniques: (1) **Residual Correction (RC)**, a zero-cost mixed-precision correction mechanism implemented as accumulator bias initialization at 8-bit precision; (2) **Multi-Adapter scaling**, which employs multiple parallel adapter branches to progressively improve task adaptation capacity; and (3) an **automated architecture search** mechanism that exhaustively evaluates adapter insertion configurations to identify the highest-accuracy combination under deployment constraints.

The hardware design integrates a custom-designed Adapter IP into FINN’s core compute unit, the MVAU, through a Super Wrapper architecture. This integration features decoupled thresholding, cycle-accurate pipeline synchronization via lightweight FIFOs, and pre-computed

lookup tables for zero-DSP contribution scaling. The complete system is implemented on the PYNQ-Z2 platform with a design flow spanning PyTorch/Brevitas training, FINN backbone generation, and custom RTL integration.

Experimental results demonstrate that: (i) the prior-work Conv-Adapter single-branch bottleneck (m1, +3.78% parameters) serves as a baseline, confirming that adapter-based transfer is feasible under 1W1A (72.12% on SVHN, 78.76% on FashionMNIST) but is insufficient on its own—we empirically observe that once the full network is quantised below 4 bits the overall contribution of a single adapter branch degrades, motivating an accuracy-recovery mechanism; (ii) this thesis successfully raises transfer accuracy under ultra-low-bit (1W1A) quantisation above the Conv-Adapter baseline, with the proposed m3 configuration (+11.35% parameters) reaching 78.97% on SVHN (+6.85 pp over the m1 baseline) and 81.13% on FashionMNIST (+2.37 pp) in software evaluation. The accuracy gain is delivered by two mechanisms introduced in this work—Multi-Adapter scaling (m2–m4) and a zero-hardware-cost Residual Correction (RC) term (RC’s standalone effect is not separately isolated in the current experiments and its inclusion is justified on principled grounds); (iii) the hardware implementation achieves 1,928 img/s end-to-end throughput (26,000 SVHN samples, 0.52 ms/img average latency) and 870.83 img/s/W energy efficiency on PYNQ-Z2 at 2.214 W total on-chip power, representing $57.6\times$ and $11.0\times$ improvements over RTX 4090 (15.12 img/s/W) and Jetson Orin NX (79.31 img/s/W), respectively; (iv) the adapter integration adds zero DSP overhead while consuming 39,493 LUTs (74.23%), 49,270 FFs (46.31%), and 99 BRAMs (70.71%), with the five adapter cores jointly occupying 25,593 LUTs ($\sim 48\%$ of the chip) fully implemented in distributed RAM and combinational logic; and (v) an AXI-Lite configuration hub allows runtime multi-task switching on a single bitstream with sub-1 ms latency, without any FPGA fabric re-configuration controller. Functional correctness is validated at three levels: per-MVAU RTL simulation (43,520 test vectors across MVAU1–MVAU5, 100% pass), top-level dataflow simulation matching the PyTorch golden reference, and end-to-end on-board verification on PYNQ-Z2.

Keywords: Parameter-Efficient Transfer Learning (PEFT), Conv-Adapter, Ultra-Low-Bit Quantization, FINN, FPGA, Multi-Adapter, Residual Correction, MVAU, Streaming Dataflow, PYNQ-Z2.

中文摘要

邊緣裝置之運算、記憶體與功耗資源先天受限 [1,31]。現行邊緣 FPGA 支援多任務 AI 推論之已發表方法主要分三類：(a) 部分重組態 (PR) 時間多工，於 Zynq 平台之實測切換延遲為 PCAP 97–114 ms [20]、ZyCAP 5.2 ms [18]，並需搭載重組態控制器；(b) 固定特徵萃取器配可程式後端 (FixyNN [39])，特徵萃取器於 tape-out 即固定、執行時期無法再適應；(c) 片上模型／分類器選擇 (NeuroAdaptiveNet [12])，所有變體須常駐晶片。三類方案皆於原始論文中各有其限制；於本論文所回顧之文獻範圍內，我們未見一套系統能同時提供次毫秒切換與特徵萃取器層級之任務適應，故本論文之貢獻宣稱僅限於此已引用之比較範圍。

近年來，參數高效率微調 (PEFT) 技術已在多個 AI 領域展現強大潛力，僅需更新少量參數即可適應新任務 [35,40]。然而，在超低位元量化下的實際硬體實作仍鮮少被探討。尤其當 CNN 主幹網路量化至 1-bit 權重與 1-bit 啟動值 (1W1A) 時，表達能力嚴重受限 [47,50]，直接將軟體端 Adapter 機制移植至硬體將同時面臨準確率下降與整合困難。

本論文提出 **Barkaic**，一種新型 AI 硬體加速器架構，同時解決超低位元 Adapter 推論之準確率與可部署性瓶頸。所提方法結合三項關鍵技術：(1) **殘差校正 (RC)**，以 8-bit 精度累加器偏置初始化實現之零成本混合精度校正機制；(2) **多 Adapter 擴展**，採用多組平行 Adapter 分支漸進提升任務適應能力；(3) **自動化架構搜尋** 機制，窮舉評估 Adapter 插入組合以在部署限制下找出最高準確率配置。

硬體設計透過新型 Super Wrapper 架構，將自研 Adapter IP 整合至 FINN 核心運算單元 MVAU。此整合採用解耦閾值處理、經由輕量 FIFO 之週期精準管線同步，以及預計算查找表實現零 DSP 貢獻量縮放。完整系統於 PYNQ-Z2 平台實作驗證，設計流程涵蓋 PyTorch/Brevitas 訓練、FINN 主幹生成與自訂 RTL 整合。

實驗結果顯示：(i) 既有之 Conv-Adapter 單分支瓶頸 (m1, +3.78% 參數，為先前工作) 作為基線，於 1W1A 下確認 Adapter 遷移可行 (SVHN 72.12%、FashionMNIST 78.76%)，但單獨不足——本研究經驗性發現：當整個網路量化至 4-bit 以下時，

單分支 Adapter 整體貢獻下滑，因而需要補救機制；(ii) 本論文成功將超低位元（1W1A）量化下之遷移準確率提升至 Conv-Adapter 基線之上——m3 配置（+11.35% 參數）於軟體端達 SVHN 78.97%（較 m1 基線 +6.85 個百分點）、FashionMNIST 81.13%（+2.37 個百分點）。此準確率提升由本論文所提之兩項機制達成：多 Adapter 擴展（m2-m4）與零硬體成本之殘差校正（RC）（目前 RC 之獨立效果尚未於實驗中單獨分離，其納入基於設計原則）；(iii) 硬體實作於 PYNQ-Z2 上達到 1,928 img/s 端到端吞吐量（26,000 張 SVHN 測試樣本，平均每張延遲 0.52 ms），於晶片總功耗 2.214 W 下能效達 870.83 img/s/W，分別為 RTX 4090（15.12 img/s/W）之 57.6 倍與 Jetson Orin NX（79.31 img/s/W）之 11.0 倍；(iv) Adapter 整合零 DSP 額外開銷，LUT 39,493（74.23%）、FF 49,270（46.31%）、BRAM 99（70.71%），五組 Adapter Core 合計使用 25,593 LUT（約佔晶片 48%），全以分散式 RAM 與組合邏輯實作；(v) 透過 AXI-Lite 配置中樞於單一 bitstream 上實現 runtime 多任務切換，切換延遲於 1 ms 以下，且無需 FPGA fabric 重組態控制器。功能正確性於三個層級驗證：各 MVAU RTL 模擬（MVAU1-MVAU5 共 43,520 筆測試向量，100% 通過）、頂層資料流模擬（與 PyTorch golden 一致），以及 PYNQ-Z2 板上端到端驗證。

關鍵詞：參數高效率遷移學習（PEFT）、Conv-Adapter、超低位元量化、FINN、FPGA、多 Adapter、殘差校正、MVAU、串流資料流、PYNQ-Z2

Contents

1	Introduction	8
1.1	Background	8
1.2	Motivation and Problem Definition	9
1.2.1	Multi-application deployment on resource-constrained edge FPGAs: published approaches and their limitations	9
1.2.2	Accuracy degradation under ultra-low-bit adapter fine-tuning	10
1.3	Method Overview	11
1.3.1	Accuracy enhancement under ultra-low-bit quantisation	11
1.3.2	Automated architecture search	11
1.3.3	Hardware design and integration	11
1.3.4	Runtime multi-task switching	12
1.4	Research Objectives	12
1.5	Contributions	12
1.6	Thesis Organization	14
2	Related Work	15
2.1	Ultra-Low-Bit FPGA CNN Accelerators and Dataflow Architectures	15
2.2	Parameter-Efficient Transfer Learning and Adapter Methods	18
2.3	Reconfigurable, Adaptive, and Shared FPGA Architectures	19
3	Methodology	23
3.1	Problem Formulation and Design Motivation	23
3.2	Binary Adapter Bottleneck Architecture	26
3.3	Residual Correction (RC)	29
3.3.1	Motivation	29

3.3.2	Zero-cost hardware implementation	29
3.3.3	Current experimental status and honest framing	30
3.4	Multi-Adapter Scaling	30
3.4.1	Architecture	30
3.4.2	Theoretical computational cost	31
3.4.3	Accuracy recovery through branch scaling	32
3.5	Automated Architecture Search	32
4	Hardware Architecture and Design	34
4.1	System Architecture Overview	34
4.2	MVAU + Adapter Super Wrapper	36
4.3	Adapter IP Internal Architecture	38
4.3.1	Down-Projection Pipeline	39
4.3.2	Up-Projection Path	39
4.3.3	Memory Organization	39
4.4	Stream Adder + Threshold Module	40
4.4.1	Contribution LUT Precomputation	40
4.4.2	Two-Stage Pipeline	41
4.5	Runtime Configuration Hub	42
4.6	Pipeline Optimization of the MVAU + Adapter Integrated System	42
4.7	End-to-End Design Flow	43
4.8	Verification Methodology	44

List of Figures

4.1	Barkaic accelerator system top-level architecture on PYNQ-Z2 (XC7Z020). . .	35
4.2	Barkaic streaming dataflow pipeline and RTL integration. Top: the complete pipeline showing FINN-generated modules (blue), custom Adapter-Enhanced Super Wrappers (orange), and auxiliary modules (gray). Bottom: expanded view of Super Wrapper internal structure with Path A (MVAU backbone) and Path B (Adapter correction) converging at the Adder + Threshold fusion point.	36
4.3	MVAU + Adapter Super Wrapper internal architecture. Path A is the unmodified MVAU backbone; Path B is the adapter correction path. Both paths converge at the Stream Adder + Threshold for fused binary output.	37
4.4	Adapter IP four-stage pipeline architecture. The down-projection path (S0–S3) processes C_h hidden channels in parallel. After sign extraction, the up-projection generates PE output values per cycle.	38
4.5	End-to-end software-to-hardware design flow for the Barkaic accelerator. . . .	44

List of Tables

2.1	Summary of key works on ultra-low-bit FPGA CNN accelerators (sorted by year, newest first)	16
2.2	Summary of key works on PEFT and adapter-based methods (sorted by year, newest first)	19
2.3	Summary of key works on reconfigurable and multi-task FPGA architectures (sorted by year, newest first)	20
2.4	Research gap analysis: positioning of Barkaic	22
3.1	Adapter bottleneck configuration per layer	28
3.2	Backbone computational cost breakdown (CIFAR-10, $32 \times 32 \times 3$ input)	32
4.1	Adapter ROM configurations (example: MVAU4, PE= 4)	40
4.2	PE/SIMD configuration per layer before and after the pipeline-aware redesign of the MVAU + Adapter integration.	43
4.3	Module-level RTL simulation results. Test vector counts correspond to per-channel outputs emitted over 10 sample inputs at each layer’s spatial output footprint.	46

List of Algorithms

3.1	Binary Adapter Forward Computation	29
3.2	Exhaustive Adapter Architecture Search	33
4.1	Stream Adder + Threshold Fusion	41

Chapter 1

Introduction

1.1 Background

Deep learning models have increasingly migrated from cloud servers to edge devices to satisfy requirements for real-time responsiveness, data privacy, and offline operation. However, edge deployment is strongly constrained by compute capability, memory capacity, and power budget, making *efficient support of multiple AI applications under limited hardware resources* a central challenge [1, 3, 31].

Research on AI hardware acceleration has evolved along multiple design routes. FPGA-based accelerators, in particular, offer reconfigurability, energy efficiency, and customizable datapaths that make them well-suited for edge inference [5, 6]. Under ultra-low-bit settings (e.g., 1-bit / 2-bit), binary and low-precision networks can reduce arithmetic complexity by replacing multiplications with XNOR and popcount operations, and rely on PE/SIMD-style parallelism to sustain throughput [38, 47, 50]. FINN is a representative toolflow for mapping quantized networks to dataflow hardware, where the Matrix-Vector-Activation Unit (MVAU) serves as the core compute building block [30, 43, 46].

Meanwhile, the demand for model adaptation is rapidly increasing. Conventional transfer learning often performs full-model fine-tuning or replaces only the classification head. As model sizes and task diversity grow, such strategies increase training cost, storage requirements, and deployment complexity [17, 48]. Parameter-Efficient Transfer Learning (PETL/PEFT) addresses this by freezing the backbone and training only a small set of task-specific parameters, thereby reducing overhead while preserving competitive performance [10, 35, 40]. In the vision domain, adapter-based approaches typically insert bottleneck modules consisting of a

down-projection, nonlinearity, and up-projection, enabling task adaptation through lightweight parameter switching [21, 29].

1.2 Motivation and Problem Definition

1.2.1 Multi-application deployment on resource-constrained edge FPGAs: published approaches and their limitations

The deployment of multiple AI applications on a single edge FPGA has been actively studied in the literature, and three dominant paradigms emerge, each with a documented limitation that this thesis directly addresses.

Partial-reconfiguration-based time multiplexing ACNNE [20] uses the PCAP interface to swap convolution layer hardware blocks at runtime, measuring 97–114 ms reconfiguration latency per partition. The coarse-to-fine approach by Rangsikunpum et al. [18] uses the faster ZyCAP controller and reports 5.2 ms per switch, which the authors note already dominates per-inference execution time. Boudjadar et al. [13] and Kang & Park [14] both rely on dynamic partial reconfiguration (DPR) and thus inherit the underlying PR-controller latency and the operational requirement of hosting a reconfiguration controller plus a partial-bitstream repository on the target device. Collectively these works confirm that sub-millisecond PR-based task switching on PYNQ-class FPGAs is not currently reported.

Fixed feature extractor, programmable back-end FixyNN [39] hardwires the feature extractor and leaves the back-end programmable, explicitly arguing that per-task full-model replication is infeasible at the edge and that the feature extractor should be shared. This is an influential framing, but it freezes the feature extractor at tape-out and therefore cannot accommodate task-specific feature adaptation, which is exactly where ultra-low-bit (1W1A) backbones lose the most accuracy.

On-chip model/classifier selection NeuroAdaptiveNet [12] stores multiple classifier variants on-chip and selects one at runtime, reporting a 52.85% LUT reduction over instantiating every variant simultaneously but still paying the resource cost of every classifier variant being always-resident.

The gap this thesis targets Across these published systems, among the cited works we did not identify one that demonstrates sub-millisecond task switching on a PYNQ-Z2-class platform without a fabric-level reconfiguration controller, while also incorporating task-specific *feature-extractor* adaptation (as opposed to classifier-only adaptation). Under 1W1A quantisation the representational capacity is too narrow for a fixed or classifier-only approach to recover the accuracy lost to binarisation, and under PR-based approaches the switching latency rules out fine-grained time-sharing. Barkaia addresses exactly this gap: task-specific conv-adapter insertion throughout the backbone (for 1W1A accuracy recovery) combined with AXI-Lite parameter-reload switching (for sub-1 ms task transitions, no reconfiguration controller needed).

1.2.2 Accuracy degradation under ultra-low-bit adapter fine-tuning

Although PEFT has been widely studied in software, its practical deployment under ultra-low-bit quantization remains challenging. Conv-Adapter [21] demonstrates that inserting a lightweight single-branch adapter while freezing the backbone can enable parameter-efficient adaptation for ConvNets; the precision redundancy analysis by Jie et al. [29] further suggests that adapters may tolerate aggressive quantization. However, when the inference backbone is implemented with 1W1A precision, the representational capacity is severely constrained, and binary network literature confirms that preserving accuracy under such settings requires specialized architectural and training strategies [37, 42, 49].

Our empirical investigation on a fully-quantised backbone further reveals that, once the precision of the whole network drops below 4 bits, the overall contribution of a single Conv-Adapter branch degrades substantially—the single-branch design inherited from prior work is no longer sufficient to absorb the representational loss induced by binarisation. This observation is the direct motivation for the two accuracy-recovery mechanisms proposed in this thesis: **Multi-Adapter scaling** (m2–m4) and **Residual Correction (RC)**. The single-adapter (m1) numbers reported later in this thesis serve as the Conv-Adapter baseline against which the proposed mechanisms are measured.

Therefore, the central question of this thesis is: *Under an ultra-low-bit FPGA inference architecture, how can multiple tasks share a single backbone with feature-extractor-level adaptation, while achieving task switching that is sub-millisecond, controller-free, and at least an order of magnitude faster than the fastest reported PR-based multi-task FPGA system?*

1.3 Method Overview

This thesis addresses both the accuracy and deployability challenges through a combination of algorithmic enhancement, automated architecture selection, and hardware design innovation.

1.3.1 Accuracy enhancement under ultra-low-bit quantisation

The accuracy-side contribution of this thesis is to successfully raise transfer accuracy under ultra-low-bit (1W1A) quantisation above the Conv-Adapter single-branch baseline [21], where the baseline itself was observed—in experiments conducted in this thesis—to lose effectiveness once the full network is quantised below 4 bits. To deliver this gain, two mechanisms are introduced: **Multi-Adapter scaling**, which replaces the single branch with $m \in \{2, 3, 4\}$ parallel adapter branches so that additional representational capacity is made available exactly where binarisation removed it; and **Residual Correction (RC)**, an 8-bit per-channel bias loaded as the accumulator initial value, contributing a zero-hardware-cost correction path. RC’s standalone effect has not been isolated experimentally in this work, and its inclusion is justified on principled grounds as a zero-overhead correction term rather than on measured benefit. The single-branch adapter (m1) is retained only as the Conv-Adapter baseline and is not claimed as a contribution of this work.

1.3.2 Automated architecture search

To avoid expensive manual trial-and-error over adapter configurations, this thesis introduces an exhaustive search mechanism that evaluates all $2^5 = 32$ possible combinations of adapter insertion positions across five convolutional layers, identifying the highest-accuracy configuration under target constraints.

1.3.3 Hardware design and integration

The core hardware contribution is a **low-power, low-resource Adapter IP** integrated into FINN’s MVAU through a **Super Wrapper** architecture. This design features decoupled thresholding, zero-DSP contribution scaling via pre-computed lookup tables, and cycle-accurate pipeline synchronization. The complete design flow spans from PyTorch/Brevitas quantization-aware training through FINN backbone generation to custom RTL integration and on-board FPGA verification [30, 46].

1.3.4 Runtime multi-task switching

Building on the adapter-based architecture, this thesis further contributes a lightweight AXI-Lite configuration hub that allows runtime task switching on a single bitstream. The hub reprograms the task-dependent thresholds, classifier weights, and adapter control registers through a unified address map, achieving sub-1 ms switching latency without invoking FPGA fabric reconfiguration.

1.4 Research Objectives

Based on the above motivations, the objectives of this thesis are:

1. Improve ultra-low-bit fine-tuning accuracy primarily through Multi-Adapter scaling, complemented by zero-cost Residual Correction, and validate across multiple transfer learning datasets.
2. Develop an automated search mechanism for adapter insertion configurations under deployment constraints.
3. Establish a controllable design flow integrating FINN-generated backbones with custom RTL Adapter modules.
4. Propose and implement validation methods targeting FINN core components (MVAU) to ensure software-to-hardware behavioral consistency.
5. Realize and verify the complete accelerator on PYNQ-Z2 with end-to-end inference capability.

1.5 Contributions

The contributions of this thesis are summarized as follows:

1. **Successfully improving transfer accuracy under ultra-low-bit quantisation:** raise 1W1A transfer accuracy above the Conv-Adapter single-branch baseline [21], which in experiments conducted as part of this thesis was observed to lose effectiveness once the full network is quantised below 4 bits. With the proposed m3 configuration (+11.35%

parameters), SVHN accuracy rises from m1 72.12% to 78.97% (+6.85 pp) and Fashion-MNIST from 78.76% to 81.13% (+2.37 pp). The gain is delivered by two mechanisms proposed in this work: *Multi-Adapter branch scaling* (m2–m4) and a zero-hardware-cost *Residual Correction (RC)* term. RC’s standalone effect is not yet isolated in the current experiments; its inclusion is justified on principled zero-overhead grounds.

2. **RTL-level low-power Adapter IP:** design an Adapter IP using only distributed RAM and combinational logic, achieving zero DSP, and implementing five per-layer adapters (MVAU1–MVAU5, Conv2–Conv6) with heterogeneous PE/SIMD/HIDDEN parameterization in a single unified RTL family. The five adapter cores jointly occupy 25,593 LUTs ($\sim 48\%$ of XC7Z020), with zero BRAM overhead through distributed-RAM ROM packing for both down- and up-projection weights.
3. **Adapter–MVAU integration via Super Wrapper:** integrate the Adapter IP into FINN’s MVAU with decoupled thresholding, pre-computed contribution LUTs, and cycle-accurate synchronization through a deterministic Simple FIFO. The Stream Adder + Threshold module fuses adapter contribution into the MVAU accumulator via a Q8 fixed-point comparison that handles both $\gamma > 0$ and $\gamma < 0$ BatchNorm polarities through a bit-reflection trick for MVAU5.
4. **Runtime multi-task switching infrastructure:** design a lightweight AXI-Lite configuration hub that reprograms the task-dependent thresholds, classifier weights, and adapter control registers on a single bitstream, achieving sub-1 ms task switching without invoking FPGA fabric reconfiguration.
5. **PYNQ-Z2 implementation and verification:** implement and validate the accelerator with three-level functional verification: (a) per-MVAU RTL simulation with a total of 43,520 test vectors achieving 100% pass, (b) top-level streaming dataflow simulation matching the PyTorch golden reference, and (c) end-to-end on-board verification on PYNQ-Z2 with 71.81% accuracy on 26,000 SVHN samples (vs. PyTorch 72.12%), 1,928 img/s end-to-end throughput, and 0.52 ms average latency per image.
6. **End-to-end design methodology:** propose a practical software-to-hardware flow spanning PyTorch/Brevitas quantization-aware training, FINN backbone IP generation, custom RTL integration, IP packaging, block design stitching, bitstream generation, and on-

board deployment, with predictable resource utilization and a documented Vivado out-of-context synthesis cache pitfall solved by the `rebuild_bitstream.tcl` build orchestrator.

7. **Automated adapter architecture search:** provide an exhaustive search mechanism over $2^5 = 32$ configurations that identifies the optimal adapter subset by zeroing the scaling factor of unused layers, allowing reuse of a single trained checkpoint without re-training.

1.6 Thesis Organization

The remainder of this thesis is organized as follows:

- **Chapter 2 (Related Work):** reviews ultra-low-bit FPGA inference, FINN/M-VAU dataflow architectures, PEFT and adapter methods for ConvNets, and multi-task/reconfigurable FPGA deployment approaches.
- **Chapter 3 (Methodology):** details the Adapter bottleneck architecture, RC mechanism, Multi-Adapter scaling strategy, and the automated architecture search, with formal algorithmic descriptions.
- **Chapter 4 (Hardware Architecture and Design):** describes the Adapter IP RTL design, Super Wrapper integration, pipeline synchronization, and the end-to-end FPGA design flow.

Chapter 2

Related Work

This chapter reviews three research lines most relevant to this thesis: ultra-low-bit FPGA CNN acceleration, parameter-efficient transfer learning (PEFT) with adapters, and reconfigurable/multi-task FPGA architectures. Each section concludes with a summary table highlighting key papers.

2.1 Ultra-Low-Bit FPGA CNN Accelerators and Dataflow Architectures

FPGA-based CNN acceleration has progressed from straightforward hardware mapping toward integrated consideration of dataflow design, low-bit quantization, and automated toolflows. Comprehensive surveys indicate that the major design issues include parallelism strategies, data reuse, memory bandwidth optimization, dataflow mapping, and energy efficiency [1, 5, 31]. Li [6] further reviews Weight-Stationary, Output-Stationary, and Row-Stationary dataflow paradigms with specific attention to FINN-style streaming architectures.

Among ultra-low-bit accelerators, FINN [46] is one of the most representative frameworks. FINN proposes a heterogeneous streaming architecture for binarized neural network inference, where hardware resources are customized layer by layer and connected via on-chip data streams. The framework was extended by FINN-R [43], which generalized the toolflow to support arbitrary-precision quantized networks. Alam et al. [30] analyzed the RTL implementation of the FINN Matrix-Vector compute Unit (MVU/MVAU), demonstrating that RTL implementations can provide shorter critical paths and better resource efficiency compared to HLS. Arend et al. [2] further proposed resource prediction models for MVAU modules in

FINN-generated accelerators. More recently, Jungemann et al. [7] extended FINN for HPC environments, achieving significant throughput improvements on large-scale FPGA platforms. Tasci et al. [8] demonstrated practical FINN+Brevitas deployment with W1A1 and W2A2 quantization on the PYNQ Z1 platform, achieving 953 kFPS at 2.05 W.

Beyond FINN, other works have explored low-bit FPGA acceleration from different angles. FP-BNN [41] demonstrated practical feasibility of binarized neural networks on FPGA. Su et al. [32] provided a comprehensive survey of BNN FPGA implementations covering multiple tool flows. Zhan et al. [33] proposed a BNN accelerator leveraging four-direction parallelism for edge computing. At the model level, XNOR-Net [47], BinaryConnect [49], Binarized Neural Networks [50], Bi-Real Net [42], and ReActNet [37] have collectively shown that maintaining accuracy under 1-bit settings requires specialized activation functions, residual connections, and training strategies. The comprehensive BNN survey by Qin et al. [38] provides a systematic overview of these techniques. At the hardware design level, streaming architectures with line-buffer organization [34] and balanced dataflow strategies [24] have further advanced throughput efficiency for lightweight CNNs. Quantization from a hardware perspective has been systematically reviewed by Fouda et al. [11] and formalized in the quantization white paper by Nagel et al. [36]. The hls4ml framework [16] represents another important open-source platform for ML-to-FPGA compilation.

Table 2.1: Summary of key works on ultra-low-bit FPGA CNN accelerators (sorted by year, newest first)

Year	Reference	Bit-width	Platform / Tool	Key Contribution
2025	hls4ml [16]	Mixed	HLS / Multi-FPGA	Open-source HLS-based ML compiler for FPGA
2025	Arend et al. [2]	1–8 bit	FINN / Zynq	Resource prediction model for MVAU modules
2025	Tasci et al. [8]	W1A1, W2A2	FINN+Brevitas / PYNQ Z1	Practical QNN deployment at 953 kFPS, 2.05 W
2025	Jungemann et al. [7]	Mixed	FINN / Alveo U250	FINN extended for HPC environments
2025	Fouda et al. [11]	Mixed	Survey	Quantization techniques from hardware perspective
2024	Zhao et al. [24]	8-bit	Custom RTL	Balanced dataflow for lightweight CNNs

continued on next page

Table 2.1 (continued)

Year	Reference	Bit-width	Platform / Tool	Key Contribution
2023	Alam et al. [30]	1–8 bit	FINN RTL / Zynq	RTL vs. HLS analysis of FINN MVU
2023	Su et al. [32]	1-bit	Survey	Comprehensive survey of BNN FPGA tool flows
2023	Zhan et al. [33]	1-bit	Custom RTL	Four-direction parallelism BNN accelerator
2023	Kang & Yang [34]	8-bit	Custom RTL	Stream-based line-buffer CNN accelerator
2020	ReActNet [37]	1W1A	Software (PyTorch)	Generalized activation for precise BNNs
2020	Qin et al. [38]	1-bit	Survey	Systematic overview of BNN techniques
2018	FINN-R [43]	Arbitrary	FINN / Multi-FPGA	Generalized FINN to arbitrary-precision QNN
2018	FP-BNN [41]	1-bit	Custom RTL / Zynq	Practical BNN implementation on FPGA
2018	Bi-Real Net [42]	1W1A	Software (PyTorch)	Real-valued shortcuts in binary networks
2017	FINN [46]	1-bit	FINN / PYNQ-Z1	Heterogeneous streaming architecture for BNN
2016	XNOR-Net [47]	1W1A	Software (Torch)	XNOR-based binary convolution for ImageNet
2015	BinaryConnect [49]	1W	Software (Theano)	Binary weights with full-precision gradients
2016	Binarized NN [50]	1W1A	Software (Theano)	Both weights and activations binarized

Despite this substantial progress, existing ultra-low-bit FPGA accelerator research remains largely focused on *single-model deployment*. The dominant assumption is that the accelerator serves one fixed model, with optimization targeting throughput, latency, and resource efficiency for that single workload. A method that incorporates task-specific adaptation into an existing low-bit datapath while preserving a shared backbone is still lacking.

2.2 Parameter-Efficient Transfer Learning and Adapter Methods

With the widespread adoption of large pre-trained models, adapting them to downstream tasks without duplicating all model parameters has become a critical research problem. The adapter paradigm was first introduced by Houlsby et al. [40] for NLP, proposing bottleneck modules inserted between Transformer layers. LoRA [35] later proposed low-rank decomposition of weight updates as an alternative PEFT strategy. The Visual Tuning survey [17] categorizes recent visual tuning methods into five groups—fine-tuning, prompt tuning, adapter tuning, parameter tuning, and remapping tuning—establishing a clear methodological landscape for vision PEFT.

For ConvNets specifically, Conv-Adapter [21] is the most relevant work. The authors argue that ConvNet intermediate representations are structured feature maps with spatial locality, and therefore adapters must preserve this locality rather than directly reusing Transformer-style designs. Conv-Adapter introduces depth-wise separable convolution-based bottleneck modules and demonstrates that approximately 3.5%–5% of backbone parameters are sufficient to match or exceed full fine-tuning performance across 23 cross-domain classification datasets. The precision redundancy analysis by Jie et al. [29] further reveals that adapters exhibit flatter loss landscapes than full fine-tuning, implying stronger robustness to quantization noise. Their experiments show that quantizing only the adapters to 1-bit causes negligible performance degradation in software, suggesting that low-bit adapters are a meaningful modeling direction.

Recent work has extended PEFT to broader contexts. LoRA-C [22] applies low-rank decomposition to convolutional layers for IoT deployment. LoRA-Edge [9] proposes tensor-train-assisted LoRA for CNN fine-tuning on edge devices such as Jetson Orin Nano, achieving $3.8\times$ faster convergence with only 1.49% trainable parameters. PEFT-Bench [10] provides a unified benchmark evaluating multiple PEFT methods with standardized cost metrics. Cache-Adapter [4] extends adapter fine-tuning to video action recognition by freezing the backbone and training only inserted adapter blocks. Liu et al. [23] survey lightweight deep learning techniques for resource-constrained environments, covering both architectural and compression approaches.

Table 2.2: Summary of key works on PEFT and adapter-based methods (sorted by year, newest first)

Year	Reference	Domain	Method Type	Trainable %	Key Contribution
2025	Cache-Adapter [4]	Video	Adapter	<5%	Adapter fine-tuning for video action recognition
2025	LoRA-Edge [9]	Vision	LoRA+TT	1.49%	Tensor-train LoRA for edge CNN fine-tuning
2025	PEFT-Bench [10]	Multi	Benchmark	Various	Unified PEFT benchmark with cost metrics
2024	Conv-Adapter [21]	Vision	Adapter	3.5–5%	ConvNet-specific adapter with depthwise conv
2024	Visual Tuning [17]	Vision	Survey	Various	Taxonomy of five visual PEFT method categories
2024	LoRA-C [22]	IoT	LoRA	<2%	Low-rank conv adaptation for IoT deployment
2024	Liu et al. [23]	General	Survey	Various	Survey of lightweight DL for constrained devices
2023	Jie et al. [29]	Vision	Adapter	<5%	Precision redundancy analysis; 1-bit adapter feasible
2022	LoRA [35]	NLP	LoRA	0.1–1%	Low-rank weight update decomposition
2019	Houlsby et al. [40]	NLP	Adapter	~3%	Original adapter bottleneck for Transformers

Despite the strong software-side evidence, the adapter-based literature shares a critical limitation: the focus remains on modeling and training methodology, without addressing how adapters should be mapped into FPGA streaming datapaths, co-designed with MVAU-style architectures, or realized on ultra-low-bit shared backbones. This gap forms one of the main motivations behind Barkaic.

2.3 Reconfigurable, Adaptive, and Shared FPGA Architectures

A third research line closely related to this thesis addresses reconfigurability and multi-task support on FPGA platforms. ACNNE [20] proposes an Adaptive CNN Engine exploiting partial

reconfiguration (PR) to dynamically reconfigure convolution layer hardware blocks at runtime. The coarse-to-fine approach by Rangsikunpum et al. [18] uses PR to switch among fine-grained classification models stored in external memory. FixyNN [39] proposes a fixed front-end feature extractor combined with a programmable back-end, recognizing the hardware value of partial backbone sharing.

More recently, ElBouazzaoui et al. [12] proposed NeuroAdaptiveNet with dynamic model selection on FPGA, achieving 52.85% LUT reduction through adaptive classifier selection. Boudjadar et al. [13] presented a co-design methodology combining CNN adaptability with dynamic PR on Xilinx ZYBO. Kang and Park [14] proposed runtime-robust edge inference with masking-based partial updates via dynamic reconfiguration. The ONNX-to-hardware flow by Manca et al. [19] further demonstrates ongoing interest in adaptive FPGA inference. Khaki and Choi [15] investigated transfer learning deployment on FPGA via Vitis-AI, and the broader edge AI deployment landscape has been surveyed by multiple recent works [3].

Table 2.3: Summary of key works on reconfigurable and multi-task FPGA architectures (sorted by year, newest first)

Year	Reference	Reconfig. Method	Platform	Key Contribution
2025	NeuroAdaptiveNet [12]	Model selection	Zynq	Dynamic classifier selection, 52.85% LUT reduction
2025	Boudjadar et al. [13]	Partial Reconfig.	ZYBO	Co-design with dynamic PR for CNN adaptation
2025	Kang & Park [14]	Masking update	Zynq	Runtime-robust edge inference via partial masking
2025	Khaki & Choi [15]	Vitis-AI	ZCU104	Transfer learning deployment on FPGA
2024	ACNNE [20]	Partial Reconfig.	Zynq	Adaptive CNN engine exploiting layer-level PR
2024	Coarse-to-Fine [18]	Partial Reconfig.	Zynq	PR-based multi-model switching for fine-grained tasks
2024	Manca et al. [19]	Design flow	Multi	ONNX-to-hardware for adaptive edge inference
2019	FixyNN [39]	Weight reload	ASIC	Fixed front-end + programmable back-end (~60/40%)

The common pattern across these works is that adaptivity is achieved through model-level or configuration-level switching—replacing entire models, reconfiguring hardware partitions,

or selecting among pre-compiled execution profiles. None of these approaches compress task-specific differences into parameter-level deltas on a shared ultra-low-bit backbone.

In addition to resource overhead, PR-based multi-task designs suffer from a fundamental *task-switching latency* problem that has received relatively little attention. Rangsikunpum et al. [18] measure a reconfiguration delay of 5.2 ms when employing ZyCAP, the highest-throughput controller available on Zynq devices. Notably, this delay already exceeds the execution time of every individual inference stage in their pipeline, meaning the reconfiguration step itself becomes the dominant time cost whenever a model change is required. Consequently, their design restricts PR to infrequent transitions between classification objectives (e.g., switching the target of a surveillance system) to avoid degrading overall throughput. The overhead is considerably worse with the more widely deployed PCAP interface: the ACNNE design by Huang et al. [20] reports configuration latencies of 97–114 ms per reconfigurable partition. These results indicate that, despite active research in PR-based edge inference, the reconfiguration delay itself remains a largely unresolved obstacle—particularly for applications that demand rapid or frequent task switching.

Barkaic addresses both shortcomings—resource scalability and task-switching latency—by entirely eliminating fabric reconfiguration. Instead, multi-task switching is realized through lightweight adapter weight reloading (~ 7.3 KB per adapter via DMA), enabling sub-millisecond task transitions without altering the FPGA logic.

Summary

Table 2.4 summarizes the positioning of Barkaic relative to the three research lines reviewed above. The key insight is that while each direction has made substantial individual progress, no existing work simultaneously addresses: (i) adapter-style PEFT on an ultra-low-bit FPGA backbone, (ii) RTL-level integration into MVAU-based streaming datapaths, and (iii) end-to-end hardware verification on a resource-constrained edge platform.

Table 2.4: Research gap analysis: positioning of Barkaic

Capability	Low-bit FPGA Accelerators	PEFT / Adapters	Reconfig. FPGA	Barkaic
Ultra-low-bit (1W1A) inference	✓	—	✓	✓
Adapter-based task representation	—	✓	—	✓
Shared backbone, parameter delta	—	✓	△	✓
MVAU/dataflow integration	✓	—	—	✓
RTL-level hardware design	✓	—	✓	✓
End-to-end FPGA verification	✓	—	✓	✓

✓: addressed; △: partially addressed; —: not addressed.

Chapter 3

Methodology

This chapter presents the algorithmic and architectural methods proposed in this thesis. Section 3.1 formalizes the problem. Section 3.2 describes the adapter bottleneck design. Section 3.3 presents the Residual Correction mechanism. Section 3.4 introduces Multi-Adapter scaling. Section 3.5 details the automated architecture search.

3.1 Problem Formulation and Design Motivation

This section traces the chain of challenges encountered during the design of Barkaic and explains how each challenge motivated a specific architectural decision.

Challenge 1: Published multi-task edge-FPGA approaches all impose controller-level costs or tape-out-level rigidity. The multi-task edge-FPGA literature has converged on three strategies—partial reconfiguration (ACNNE 97–114 ms with PCAP [20], CoarseToFine 5.2 ms with ZyCAP [18], Boudjadar [13], Kang & Park [14]), fixed-feature-extractor with programmable back-end (FixyNN [39]), and on-chip classifier-variant selection (NeuroAdaptiveNet [12]). Each of these comes with a specific operational price documented in the respective paper: PR-family works require hosting a reconfiguration controller plus a partial-bitstream repository and incur the above latencies; FixyNN freezes its feature extractor at tape-out and cannot perform feature-level task adaptation; and NeuroAdaptiveNet keeps every classifier variant always-resident on-chip. The design target for Barkaic, motivated by these documented costs, is to combine feature-extractor-level task adaptation (needed to recover accuracy under 1W1A, a known hard regime [47, 50]) with sub-millisecond, controller-free

task switching.

Challenge 2: No mature adapter hardware exists. Although PEFT adapters have been extensively studied in software (LoRA [35], Conv-Adapter [21], Houshy adapters [40]), the literature reveals a critical gap: no existing work provides a mature, verified RTL implementation of adapter modules integrated into FPGA streaming datapaths. Existing FPGA accelerator research focuses on single-model deployment and does not address how adapter bottleneck operations should be mapped onto hardware processing elements. Therefore, Barkaia must design custom adapter hardware from scratch.

Challenge 3: The single-adapter baseline loses effectiveness below 4 bits. Building custom hardware for edge FPGA demands aggressive quantization to minimize resource usage. While 1-bit weight and 1-bit activation (1W1A) quantization is well-established for backbone CNNs [47, 50], applying the same ultra-low-bit quantization to a single Conv-Adapter branch (m1) is not enough: an empirical investigation conducted as part of this thesis shows that once the precision of the whole network drops below 4 bits, the overall contribution of a single adapter branch degrades substantially. The single-branch design inherited from Conv-Adapter [21] is therefore retained only as a baseline; a dedicated accuracy-recovery mechanism is required before the adapter path can be committed to ultra-low-bit hardware.

Solution: Multi-Branch Adapter with Residual Correction. Barkaia successfully raises accuracy above the Conv-Adapter baseline under this sub-4-bit regime by introducing **Multi-Branch Adapter (MB-Adapter)** scaling: instead of the single Conv-Adapter branch [21] (m1, prior work), $m \in \{2, 3, 4\}$ parallel adapter branches are summed before thresholding. Each branch learns a different correction subspace; increasing m progressively restores the representational capacity lost under sub-4-bit quantisation, with a linear and predictable resource growth (Section 3.4). Multi-Adapter scaling is the mechanism through which the accuracy-side contribution of this thesis is delivered.

As a complementary mechanism, Barkaia also incorporates **Residual Correction (RC)**: an 8-bit per-channel bias loaded as the accumulator initial value before binary popcount accumulation begins. RC requires zero additional hardware—no extra adders, DSP slices, or pipeline stages—since the bias is absorbed into the existing accumulator initialization logic (Section 3.3). In the current experiments RC’s standalone benefit is not separately isolated (m1 with and with-

out RC produces the same numbers); we therefore do not claim an independent accuracy gain for RC, and its inclusion is justified on the principled ground that it adds a per-channel correction path at zero hardware cost.

Challenge 4: Full-custom RTL is too slow for the backbone. Writing the entire accelerator—backbone and adapter—in hand-crafted Verilog would be prohibitively time-consuming. The FINN framework [43, 46] provides an automated pipeline that compiles a quantized PyTorch model (via Brevitas [45]) into a streaming MVAU-based datapath, generating synthesizable RTL for the backbone in minutes rather than months.

Challenge 5: FINN does not support adapter generation. While FINN efficiently generates the backbone datapath, it has no built-in support for adapter modules. The FINN compiler cannot produce adapter bottleneck RTL, and its generated MVAU wrappers do not expose the internal signals (accumulator values, cycle counters) needed for adapter integration. Moreover, directly modifying FINN-generated RTL is fragile: FINN’s automated pipeline overwrites outputs on each recompilation, and the generated code lacks human-readable structure for safe manual editing.

Barkaic’s solution: Hybrid FINN + custom RTL integration flow. Barkaic resolves this by proposing a hybrid design flow. The backbone MVAU RTL is generated by FINN, while the adapter modules, stream splitter, stream adder, FIFO buffers, and the enclosing Super Wrapper are implemented as hand-crafted Verilog. A custom integration script merges the two: it extracts the FINN-generated MVAU IP, wraps it inside the Super Wrapper that routes data through both the backbone and adapter paths, and produces a unified top-level design ready for Vivado synthesis. This flow preserves FINN’s productivity for the backbone while enabling full control over adapter timing, adapter weight storage, and multi-branch accumulation at the RTL level (see Chapter 4 for implementation details).

Formal objective. With the adapter approach, the resource model becomes:

$$R_{\text{proposed}}(T) = R_{\text{backbone}} + T \cdot \Delta R_{\text{adapter}}, \quad (3.1)$$

where R_{backbone} is the shared backbone cost and $\Delta R_{\text{adapter}}$ is the per-task adapter increment. Since $\Delta R_{\text{adapter}} \ll R_{\text{model}}$, the growth rate is significantly reduced. The optimization objective is:

$$\max_{\phi^{(t)}} \text{Acc}(\mathcal{M}(\theta_{\text{bb}}, \phi^{(t)}); \mathcal{D}_t^{(t)}) \quad \text{s.t.} \quad R_{\text{proposed}}(T) \leq R_{\text{max}}, \quad (3.2)$$

where $\phi^{(t)}$ denotes the task-specific adapter parameters for task t .

3.2 Binary Adapter Bottleneck Architecture

The adapter module follows a bottleneck design inspired by Conv-Adapter [21], adapted for ultra-low-bit hardware implementation. Given an input feature map with C_{in} channels, the adapter performs three sequential operations:

Down-Projection. The input activation is projected from C_{in} channels to a reduced hidden dimension C_h , computed as:

$$C_h = \left\lfloor \frac{C_{\text{in}}}{r} \right\rfloor, \quad (3.3)$$

where r denotes the reduction ratio, set to $r = 4$ in all Barkaic configurations. For each hidden unit j (ranging from 0 to $C_h - 1$), the down-projection accumulates binary inner products over K input chunks, with the RC bias as initial value:

$$h_j = b_j + \sum_{k=0}^{K-1} \text{popcount}(\text{XNOR}(\mathbf{w}_{j,k}, \mathbf{x}_k)). \quad (3.4)$$

Each symbol is defined as follows:

- h_j : the accumulated hidden value for hidden channel j , stored in a 16-bit signed register.
- b_j : the 8-bit signed RC bias for channel j (see Section 3.3), loaded as the accumulator initial value before any popcount accumulation begins.
- $K = C_{\text{in}}/S$: the number of input chunks, determined by dividing the input channel count by the SIMD width S (typically $S = 32$).
- $\mathbf{w}_{j,k} \in \{0, 1\}^S$: the S -bit binary down-projection weight vector for hidden channel j and input chunk k , stored in distributed ROM (`rom_down`).
- $\mathbf{x}_k \in \{0, 1\}^S$: the k -th chunk of the input activation vector.

- $\text{XNOR}(\cdot, \cdot)$: bitwise exclusive-NOR, producing a S -bit vector where matching bits yield 1.
- $\text{popcount}(\cdot)$: counts the number of 1-bits in the result, yielding a value in $[0, S]$.

The XNOR-popcount primitive is a standard binary neural network operation [47, 50]. Barkaie’s contribution lies in how this primitive is organized: the accumulator is initialized with the RC bias b_j rather than zero, enabling mixed-precision correction without additional hardware (Section 3.3).

Binary Activation. The hidden representation is binarized via sign extraction:

$$a_j = \begin{cases} 1 & \text{if } h_j \geq 0, \\ 0 & \text{otherwise.} \end{cases} \quad (3.5)$$

This operation requires only the MSB of each hidden accumulator, avoiding dedicated comparator logic.

Up-Projection. After all input chunks are processed and the hidden activation vector $\mathbf{a} \in \{0, 1\}^{C_h}$ is formed, it is projected back to the output channel dimension. For each processing element (PE) p (ranging from 0 to $P - 1$), the output at step s is:

$$o_{s,p} = \text{popcount}(\text{XNOR}(\mathbf{a}, \mathbf{w}_{s,p}^{\text{up}})), \quad (3.6)$$

where:

- $o_{s,p}$: the 8-bit unsigned output popcount for PE p at output step s .
- $\mathbf{a} \in \{0, 1\}^{C_h}$: the binarized hidden activation vector produced by sign extraction.
- $\mathbf{w}_{s,p}^{\text{up}} \in \{0, 1\}^{C_h}$: the up-projection weight vector for step s and PE p , stored in distributed ROM (`rom_up`).
- P : the number of PEs (processing elements), determining how many output channels are computed per cycle.
- The total number of output steps is C_{out}/P , where C_{out} is the output channel count.

In the RTL implementation, all P weight vectors for a given step are packed into a single wide ROM word (e.g., $P \times C_h$ bits), enabling single-port ROM access per output step.

Spatial Center Selection. For 3×3 convolutional layers, the adapter operates on the center pixel of each 3×3 sliding window rather than all nine positions. This is a deliberate design choice specific to Barkaic: since the FINN MVAU streams the nine kernel positions sequentially (one input chunk per cycle), the adapter only needs to process the center position (position 4 of 0–8), reducing the adapter’s computational load by $9\times$. The center pixel is identified by a cycle counter:

$$\text{center active} \iff 4K \leq \text{cycle_cnt} < 5K, \quad (3.7)$$

where $K = C_{\text{in}}/S$ is the number of chunks per kernel position. The adapter FSM gates the accumulation accordingly, producing valid output only after the center pixel chunks have been processed.

Table 3.1 lists the adapter configuration for each convolutional layer.

Table 3.1: Adapter bottleneck configuration per layer

Layer	C_{in}	C_{out}	r	C_h	PE	SIMD
Conv1 (MVAU0)	3	64	–	–	16	3
Conv2 (MVAU1)	64	64	4	16	32	32
Conv3 (MVAU2)	64	128	4	16	16	32
Conv4 (MVAU3)	128	128	4	32	16	32
Conv5 (MVAU4)	128	256	4	32	4	32
Conv6 (MVAU5)	256	256	4	64	1	32

Conv1 is excluded from adapter insertion as it performs generic low-level feature extraction [21].

The complete adapter forward computation is summarized in Algorithm 3.1.

The single-adapter parameter overhead is:

$$\Delta P = \sum_{l=1}^5 (C_{\text{in}}^{(l)} \cdot C_h^{(l)} + C_h^{(l)} \cdot C_{\text{out}}^{(l)} + C_h^{(l)}) = 58,533, \quad (3.8)$$

representing $\Delta P/P_{\text{backbone}} = 58,533/1,546,690 \approx 3.78\%$ of the backbone parameters.

Algorithm 3.1 Binary Adapter Forward Computation

Require: Input stream $\mathbf{x}^{(k)}$ for $k = 0, \dots, K - 1$; weights $\mathbf{W}_{\text{down}}, \mathbf{W}_{\text{up}}$; RC bias $\mathbf{b}_{\text{RC}}$

Ensure: Output popcount vector $\mathbf{o}$ of dimension P per output step

```
1: // Phase 1: Down-Projection (iterate over input chunks)
2: for  $j = 0$  to  $C_h - 1$  do
3:    $h_j \leftarrow b_{\text{RC}}^{(j)}$  ▷ Initialize with RC bias
4: end for
5: for  $k = 0$  to  $K - 1$  do ▷ Only center pixel chunks
6:   for  $j = 0$  to  $C_h - 1$  do
7:      $h_j \leftarrow h_j + \text{popcount}(\text{XNOR}(\mathbf{w}_{\text{down}}^{(j,k)}, \mathbf{x}^{(k)}))$ 
8:   end for
9: end for
10: // Phase 2: Binary Activation
11: for  $j = 0$  to  $C_h - 1$  do
12:    $a_j \leftarrow \mathbf{1}[h_j \geq 0]$  ▷ Sign extraction
13: end for
14: // Phase 3: Up-Projection (iterate over output steps)
15: for  $s = 0$  to  $C_{\text{out}}/P - 1$  do
16:   for  $p = 0$  to  $P - 1$  do
17:      $o_p \leftarrow \text{popcount}(\text{XNOR}(\mathbf{a}, \mathbf{w}_{\text{up}}^{(s,p)}))$ 
18:   end for
19:   emit( $o_0, o_1, \dots, o_{P-1}$ )
20: end for
```

3.3 Residual Correction (RC)

3.3.1 Motivation

Under 1W1A quantization, the accumulator’s dynamic range is inherently limited by the popcount output range. For a SIMD width of $S = 32$, each popcount produces a value in $[0, 32]$, and after K accumulation steps the total range is $[0, 32K]$. This limited range constrains the adapter’s ability to distinguish fine-grained differences between task-specific features.

The Residual Correction (RC) mechanism addresses this by introducing a per-channel 8-bit signed bias term $b_{\text{RC}}^{(j)} \in [-128, 127]$ that is loaded as the accumulator’s initial value *before* the popcount accumulation begins:

$$h_j = b_{\text{RC}}^{(j)} + \sum_{k=0}^{K-1} \text{popcount}(\text{XNOR}(\mathbf{w}_{\text{down}}^{(j,k)}, \mathbf{x}^{(k)})). \quad (3.9)$$

3.3.2 Zero-cost hardware implementation

The key insight of RC is that it requires *zero additional hardware resources*. In the RTL implementation, the accumulator register `hidden_acc[j]` is initialized to the signed RC bias

$\text{rom_rc}[j]$ during the reset phase or at the start of each new window:

$$\text{hidden_acc}[j] \stackrel{\text{init}}{\leftarrow} \text{rom_rc}[j], \quad j = 0, \dots, C_h - 1. \quad (3.10)$$

No additional adders, DSP slices, or pipeline stages are introduced. The RC values are stored in a small distributed ROM (rom_rc , $C_h \times 16$ bits), and the initialization is absorbed into the existing reset/window-start control logic. This makes RC a mixed-precision correction mechanism that operates at effectively zero marginal cost—the 8-bit correction is fused into the 1-bit accumulation pathway without any additional arithmetic hardware.

3.3.3 Current experimental status and honest framing

In the single-adapter (m1) configuration, the experimental results show that RC and non-RC variants achieve identical accuracy (72.05% on SVHN, 78.76% on FashionMNIST), so on the current evidence we *do not* claim an independent accuracy gain for RC. The interaction between RC and multi-adapter scaling (m2–m4) has not been separately ablated in this work either; the reported m2/m3/m4 numbers are all obtained with RC enabled, but we cannot yet attribute a specific portion of those gains to RC in isolation.

RC is nevertheless proposed here on principled grounds: (i) it provides an additional per-channel correction pathway, and (ii) its hardware cost is effectively zero—no extra adders, DSP slices, or pipeline stages—since it is absorbed into the existing accumulator initialisation logic. A controlled ablation that isolates RC’s contribution under m2/m3/m4 is left as future work.

3.4 Multi-Adapter Scaling

3.4.1 Architecture

Multi-Adapter scaling is the mechanism through which this thesis raises transfer accuracy above the Conv-Adapter single-branch baseline [21] under ultra-low-bit quantisation. It employs M parallel adapter branches within each layer, directly targeting the degradation of the single-branch adapter contribution that we observe once full-network quantisation drops below 4 bits. Each branch has independent weights and RC biases. At layer l , the combined adapter output

is:

$$\mathbf{y}_l = \sum_{m=1}^M \alpha_m \cdot f_m(\mathbf{x}_l), \quad (3.11)$$

where:

- $\mathbf{y}_l$: the aggregated adapter output at layer l .
- M : the number of parallel adapter branches (m1 through m4 correspond to $M \in \{1, 2, 3, 4\}$).
- α_m : a learnable scaling factor for the m -th branch, trained jointly during fine-tuning. At deployment, α_m is quantized and absorbed into the contribution LUT.
- $f_m(\mathbf{x}_l)$: the m -th adapter branch output, computed by the down-projection, binary activation, and up-projection pipeline described in Section 3.2.
- $\mathbf{x}_l$: the input feature map at layer l , shared across all branches.

The total parameter overhead scales linearly with M :

$$\Delta P(M) = M \times 58,533, \quad (3.12)$$

yielding $\Delta P/P_{\text{backbone}} \approx 3.78\% \times M$. The configurations m1 through m4 correspond to $M \in \{1, 2, 3, 4\}$.

3.4.2 Theoretical computational cost

The computational cost of the backbone and adapter branches is analyzed as follows. The backbone consists of six 3×3 convolutional layers and three fully connected layers:

Table 3.2: Backbone computational cost breakdown (CIFAR-10, $32 \times 32 \times 3$ input)

Layer	Output Shape	MACs
Conv1	$30 \times 30 \times 64$	1,555,200
Conv2	$28 \times 28 \times 64$	28,901,376
Conv3	$12 \times 12 \times 128$	10,616,832
Conv4	$10 \times 10 \times 128$	14,745,600
Conv5	$3 \times 3 \times 256$	2,654,208
Conv6	$1 \times 1 \times 256$	589,824
FC1–FC3	512, 512, 10	398,336
Total		59.46 M MACs

Each adapter branch employs 1×1 down- and up-projections across five layers, totaling 4.23 M MACs per branch. The unified computation formula is:

$$\text{OPs}_{\text{total}} = 2 \times (59.46 + 4.23 \times M) \text{ M OPs/image}, \quad (3.13)$$

where we define $1 \text{ MAC} = 2 \text{ OPs}$. For $M = 1$, the total is 127.38 M OPs/image.

3.4.3 Accuracy recovery through branch scaling

The primary motivation for Multi-Adapter scaling is to progressively recover accuracy lost due to ultra-low-bit quantization of the adapter. Increasing M expands the representational capacity of the adapter module by allowing each branch to learn a different correction subspace. Experimental evaluation of the scaling behaviour and the diminishing returns beyond three branches (consistent with the adapter bottleneck dimension limiting total information capacity) is documented separately; this chapter focuses on the methodology.

3.5 Automated Architecture Search

Given that adapters are inserted at five convolutional layers (Conv2–Conv6), the search space comprises $2^5 = 32$ possible configurations, where each layer independently includes or excludes its adapter. For each configuration $\mathbf{s} = (s_1, \dots, s_5) \in \{0, 1\}^5$, adapter layers with $s_l = 0$ are

deactivated by setting their scaling factors to zero:

$$\alpha_m^{(l)} \leftarrow 0 \quad \forall m, \quad \text{if } s_l = 0. \quad (3.14)$$

The search procedure evaluates accuracy for all 32 configurations on the target validation set and selects the Pareto-optimal point considering accuracy versus parameter count:

$$\mathbf{s}^* = \arg \max_{\mathbf{s} \in \{0,1\}^5} \text{Acc}(\mathcal{M}(\theta_{\text{bb}}, \phi_{\mathbf{s}}); \mathcal{D}_{\text{val}}^{(t)}). \quad (3.15)$$

Algorithm 3.2 formalizes this procedure.

Algorithm 3.2 Exhaustive Adapter Architecture Search

Require: Trained model with all adapters; target validation set $\mathcal{D}_{\text{val}}$

Ensure: Optimal configuration $\mathbf{s}^*$ and corresponding accuracy

```

1: best_acc  $\leftarrow$  0;  $\mathbf{s}^* \leftarrow \mathbf{0}$ 
2: for each  $\mathbf{s} \in \{0, 1\}^5$  do ▷ 32 configurations
3:   Load full model weights
4:   for  $l = 1$  to 5 do
5:     if  $s_l = 0$  then
6:       Set  $\alpha_m^{(l)} \leftarrow 0$  for all branches  $m$  ▷ Deactivate layer  $l$ 
7:     end if
8:   end for
9:   acc  $\leftarrow$  Evaluate( $\mathcal{M}$ ;  $\mathcal{D}_{\text{val}}$ )
10:  params  $\leftarrow$  CountEffectiveParams( $\mathcal{M}$ ,  $\mathbf{s}$ )
11:  if acc > best_acc then
12:    best_acc  $\leftarrow$  acc;  $\mathbf{s}^* \leftarrow \mathbf{s}$ 
13:  end if
14: end for
15: return  $\mathbf{s}^*$ , best_acc

```

This exhaustive search is computationally tractable because: (i) the search space is small ($2^5 = 32$); (ii) each evaluation requires only a forward pass over the validation set; and (iii) no retraining is needed since deactivation is achieved by zeroing the scaling factors.

Chapter 4

Hardware Architecture and Design

This chapter describes the RTL-level hardware design of the Barkaic accelerator. Section 4.1 presents the system-level architecture. Section 4.2 details the Super Wrapper that integrates the Adapter IP into the MVAU. Section 4.3 describes the Adapter IP's internal pipeline. Section 4.4 explains the Stream Adder + Threshold module. Section 4.7 outlines the end-to-end design flow.

4.1 System Architecture Overview

The Barkaic accelerator is implemented on the Xilinx Zynq-7020 SoC (PYNQ-Z2 platform), which integrates an ARM Cortex-A9 Processing System (PS) with Programmable Logic (PL). The system operates at 100 MHz (FCLK_CLK0).

Figure 4.1 shows the top-level architecture. The PS handles system control, data movement, and inference flow management via DDR3 memory. The PL implements the streaming dataflow inference pipeline, comprising:

- **Input/Output DMA:** AXI-Stream interfaces for data transfer between PS and PL.
- **Conv1 MVAU:** Unmodified FINN-generated MVAU for the first convolutional layer.
- **Conv2–Conv6 Super Wrappers:** Five adapter-enhanced MVAU modules with custom RTL.
- **FC1–FC3 MVAUs:** FINN-generated fully connected layers.
- **MaxPool $\times$ 2:** Pooling modules between convolutional stages.
- **LabelSelect:** Final classification output.

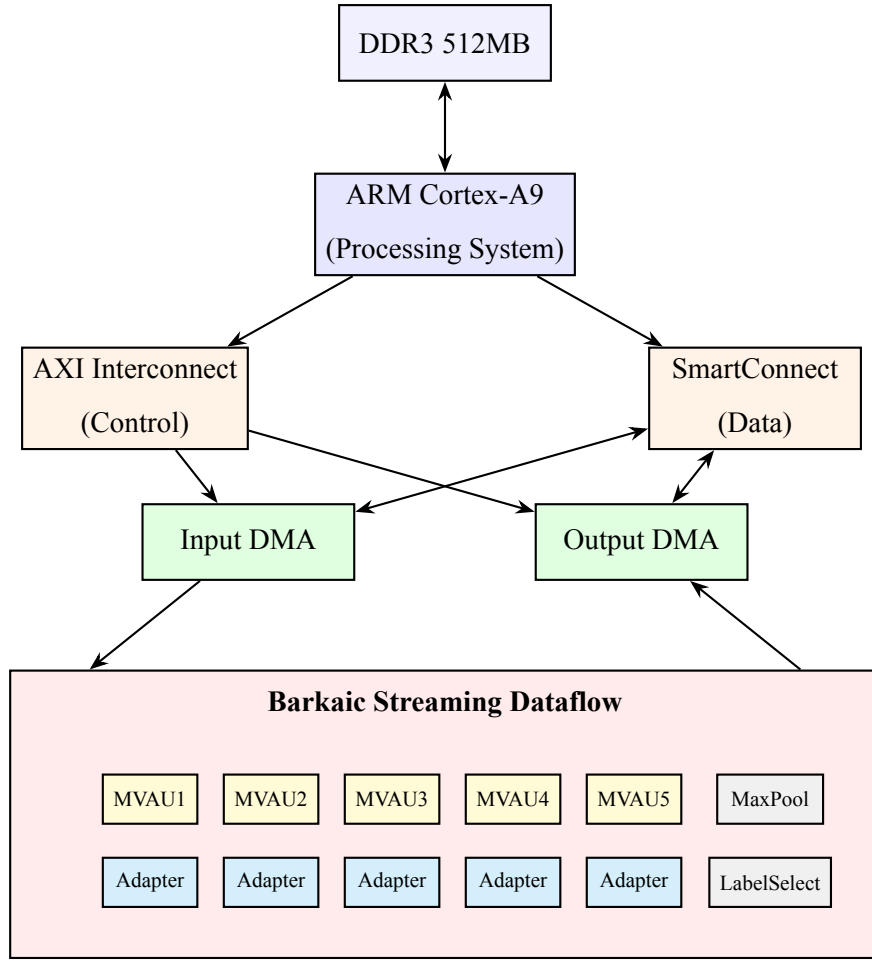
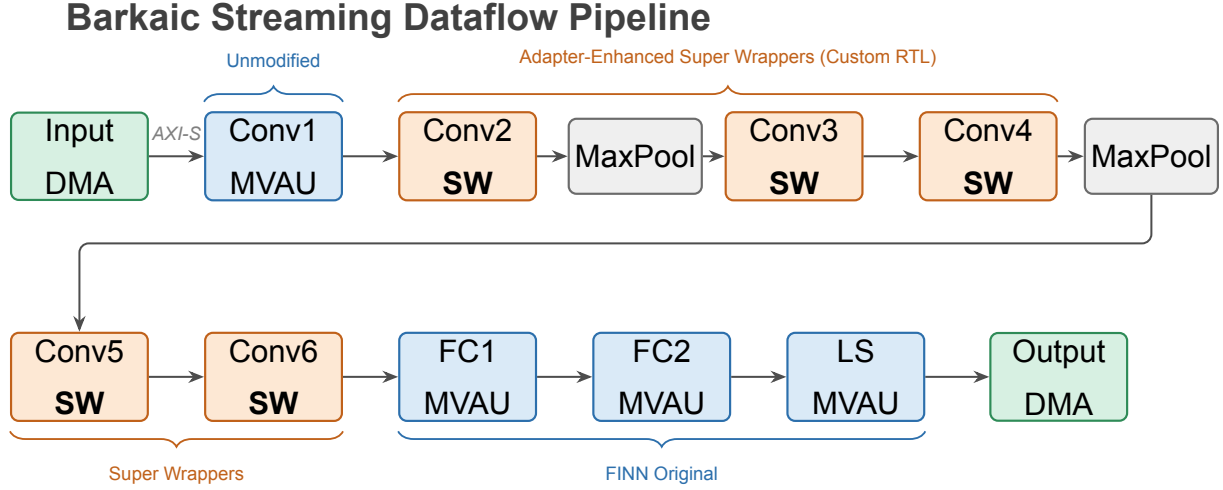


Figure 4.1: Barkaic accelerator system top-level architecture on PYNQ-Z2 (XC7Z020).

The streaming dataflow pipeline is organized as a two-row layout: the first row contains Input DMA → Conv1 MVAU → Conv2–Conv4 Super Wrappers → MaxPool; the second row continues with Conv5–Conv6 Super Wrappers → FC layers → Output DMA. All inter-module connections use AXI-Stream protocol.

Figure 4.2 provides a detailed view of the complete pipeline, distinguishing FINN-generated modules (blue) from custom Adapter-Enhanced Super Wrappers (orange), and includes an expanded view of the Super Wrapper internal structure.



Super Wrapper Internal Structure

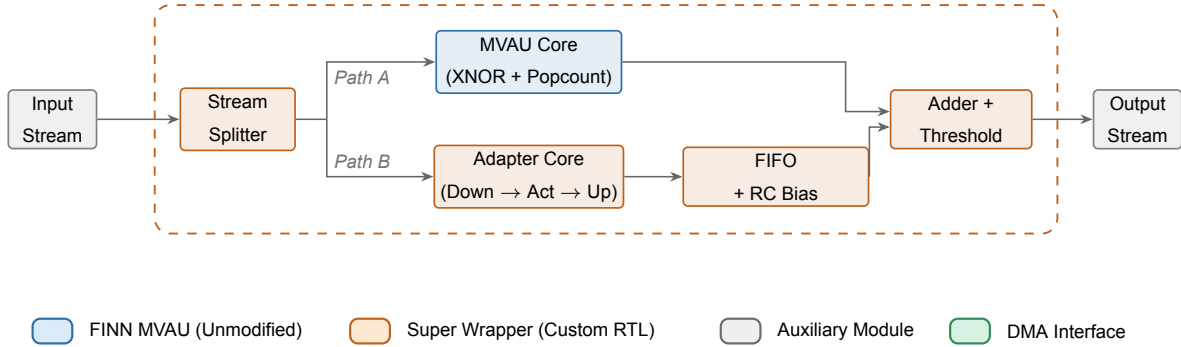


Figure 4.2: Barkaic streaming dataflow pipeline and RTL integration. Top: the complete pipeline showing FINN-generated modules (blue), custom Adapter-Enhanced Super Wrappers (orange), and auxiliary modules (gray). Bottom: expanded view of Super Wrapper internal structure with Path A (MVAU backbone) and Path B (Adapter correction) converging at the Adder + Threshold fusion point.

4.2 MVAU + Adapter Super Wrapper

The Super Wrapper is the core architectural innovation of Barkaic. It replaces each MVAU module (Conv2–Conv6) with an integrated unit that computes both the backbone MVAU output and the adapter correction in parallel.

Figure 4.3 shows the internal architecture of a Super Wrapper.

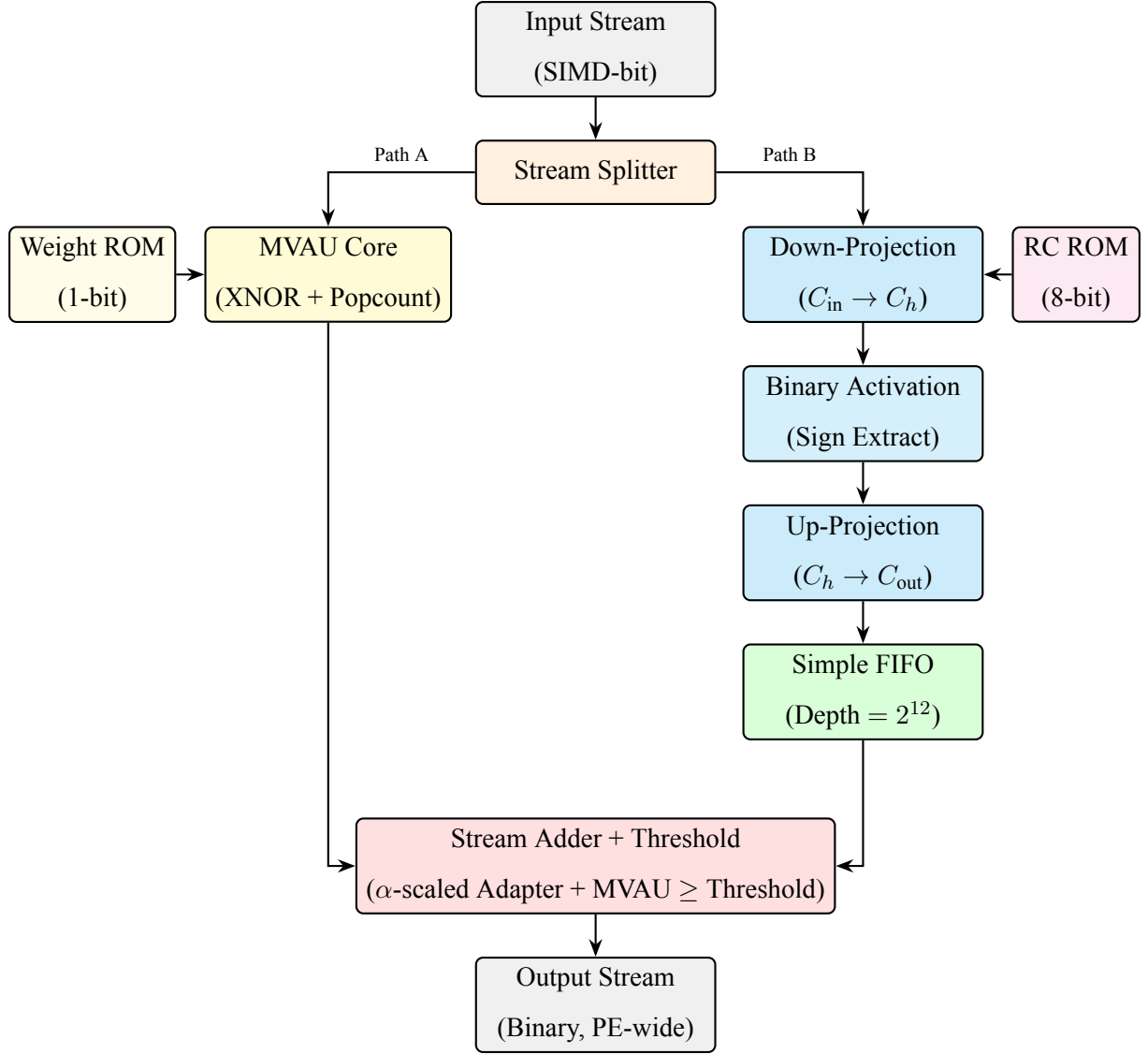


Figure 4.3: MVAU + Adapter Super Wrapper internal architecture. Path A is the unmodified MVAU backbone; Path B is the adapter correction path. Both paths converge at the Stream Adder + Threshold for fused binary output.

The Super Wrapper operates as follows:

1. **Stream Splitter**: Replicates the input stream to both paths using combinational logic. Both outputs must be ready before accepting input:

$$s_axis_tready = m_axis_1_tready \mathbf{AND} m_axis_2_tready. \quad (4.1)$$

2. **Path A — MVAU Core**: The original FINN-generated MVAU core is modified to output raw partial sums (integer MAC results) *before* thresholding. The internal threshold/binarization logic is decoupled from the MVAU and moved to the Stream Adder + Threshold

module. This allows the adapter contribution to be fused with the MVAU result before the final binary decision.

3. **Path B — Adapter Core:** Executes the adapter forward computation (Algorithm 3.1) using the dedicated Adapter IP (Section 4.3).
4. **Simple FIFO:** A lightweight synchronous FIFO (depth = $2^{12} = 4,096$) provides phase alignment between the MVAU and adapter paths, which have different internal pipeline depths. The FIFO absorbs the latency mismatch and ensures cycle-accurate synchronization at the fusion point.
5. **Stream Adder + Threshold:** Fuses the MVAU partial sums with the adapter contribution and performs the final threshold comparison to produce binary output (Section 4.4).

4.3 Adapter IP Internal Architecture

The Adapter IP implements a four-stage pipelined architecture for the down-projection path, followed by a two-stage output path for the up-projection. Figure 4.4 illustrates the pipeline structure.

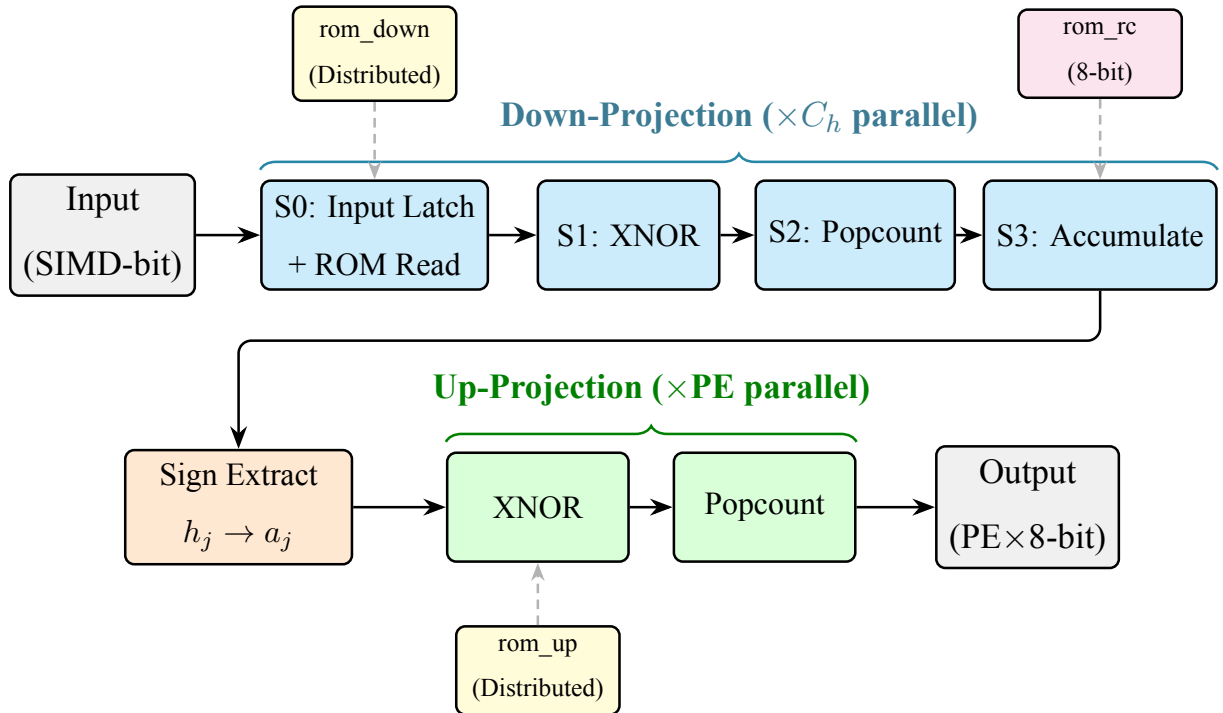


Figure 4.4: Adapter IP four-stage pipeline architecture. The down-projection path (S0–S3) processes C_h hidden channels in parallel. After sign extraction, the up-projection generates PE output values per cycle.

4.3.1 Down-Projection Pipeline

The down-projection path processes the center pixel of each 3×3 window through four pipeline stages:

- **S0 (Input Latch + ROM Read):** Captures the input activation and reads the corresponding down-projection weights from `rom_down` for all C_h hidden channels simultaneously.
- **S1 (XNOR):** Computes $\text{XNOR}(s1_act, s1_down[j])$ for each hidden channel j .
- **S2 (Popcount):** Counts the set bits of each XNOR result using a combinational adder tree synthesized from:

```
popcount_simd = 0;
for (n = 0; n < SIMD; n = n + 1)
    popcount_simd = popcount_simd + vec[n];
```

- **S3 (Accumulate):** Adds the popcount result to the running accumulator, which was initialized with the RC bias value.

4.3.2 Up-Projection Path

After all input chunks are processed, the binarized hidden activation vector (`hidden_act`) drives the up-projection:

1. The up-projection weights are packed into wide ROM words. For $PE = 4$, four 32-bit weight vectors are concatenated into a single 128-bit word, enabling single-port ROM access:

$$\text{rom_up}[s] = \{\mathbf{w}_{\text{up}}^{(s,3)}, \mathbf{w}_{\text{up}}^{(s,2)}, \mathbf{w}_{\text{up}}^{(s,1)}, \mathbf{w}_{\text{up}}^{(s,0)}\}. \quad (4.2)$$

2. XNOR is computed between `hidden_act` and each PE's weight slice.
3. Popcount produces the 8-bit output per PE.

4.3.3 Memory Organization

All adapter weights use distributed RAM (LUT RAM) with the Verilog synthesis attribute (`*rom_style = "distributed" *`). This design choice:

- Provides single-cycle read latency without BRAM pipeline overhead.
- Avoids consuming BRAM resources, which are reserved for the backbone MVAU weight storage.
- Is feasible because the adapter weights are 1-bit, resulting in compact total storage (58,533 bits per adapter).

The specific ROM configurations are summarized in Table 4.1.

Table 4.1: Adapter ROM configurations (example: MVAU4, PE= 4)

ROM	Depth	Width	Contents
rom_down	$C_h \times K = 128$	32 bits	Down-projection weights
rom_up	$C_{\text{out}}/P = 64$	$P \times 32 = 128$ bits	Packed up-projection weights
rom_rc	$C_h = 32$	16 bits	RC bias (signed)

4.4 Stream Adder + Threshold Module

The Stream Adder + Threshold module fuses the MVAU partial sums with the adapter contribution and produces binary output. Its design achieves zero DSP usage through pre-computed lookup tables.

4.4.1 Contribution LUT Precomputation

The adapter output (popcount value o_p) must be scaled by a quantized scaling factor α_{Q8} before addition to the MVAU output. Rather than performing runtime multiplication, a 256-entry lookup table is pre-computed at synthesis time:

$$\text{LUT}[i] = \left\lfloor \frac{(i - z) \times \alpha_{Q8} + 128}{256} \right\rfloor, \quad i = 0, \dots, 255, \quad (4.3)$$

where z is the zero-point (equal to $C_h/2$ for the popcount range $[0, C_h]$). The Verilog implementation uses an `initial` block to compute all 256 entries as synthesis-time constants:

```
initial begin
    for (i = 0; i < 256; i = i + 1) begin
        temp_raw = i - ADP_ZERO_POINT;
```

```

    temp_prod = temp_raw * ALPHA_Q8;
    temp_round = temp_prod + 64'sd128;
    adp_contrib_lut[i] = temp_round >>> 8;
end
end

```

This eliminates all runtime multiplications and ensures zero DSP consumption.

4.4.2 Two-Stage Pipeline

The module operates as a two-stage pipeline:

Stage 0 (LUT Lookup + Input Capture): When both MVAU and adapter outputs are valid:

$$\text{contrib}_p = \text{LUT}[\text{s_axis_2_tdata}[(p \times 8) + : 8]], \quad p = 0, \dots, P - 1. \quad (4.4)$$

Stage 1 (Addition + Threshold): The total activation is computed and compared against the per-PE threshold:

$$\text{total}_p = \text{mvau_pop}_p + \text{contrib}_p, \quad (4.5)$$

$$\text{out}_p = \begin{cases} 1 & \text{if } \text{total}_p \geq \text{thresh}_p[s], \\ 0 & \text{otherwise,} \end{cases} \quad (4.6)$$

where $\text{thresh}_p[s]$ is the threshold value for PE p at output step s , stored in per-PE threshold ROMs.

Algorithm 4.1 summarizes the complete fusion process.

Algorithm 4.1 Stream Adder + Threshold Fusion

Require: MVAU output $\mathbf{m} \in \mathbb{Z}^P$; Adapter popcount $\mathbf{o} \in [0, C_h]^P$; Threshold ROM; Contribution LUT

Ensure: Binary output $\mathbf{b} \in \{0, 1\}^P$

```

1: // Stage 0: LUT Lookup
2: for  $p = 0$  to  $P - 1$  do
3:    $\text{contrib}_p \leftarrow \text{LUT}[\mathbf{o}_p]$  ▷ Pre-computed at synthesis
4: end for
5: // Stage 1: Addition + Threshold
6: for  $p = 0$  to  $P - 1$  do
7:    $\text{total}_p \leftarrow m_p + \text{contrib}_p$ 
8:    $b_p \leftarrow \mathbf{1}[\text{total}_p \geq \text{thresh}_p[\text{step}]]$ 
9: end for
10: return  $\mathbf{b}$ 

```

4.5 Runtime Configuration Hub

To allow a single compiled bitstream to serve multiple downstream tasks at runtime, Barkaïc inserts a lightweight AXI-Lite configuration hub into the streaming dataflow partition. When two downstream tasks share the same 1W1A CNV backbone, the binary convolution and FC weights are identical; only the BatchNorm-fused thresholds, the classifier output-projection weights, and the adapter enable/threshold/sign registers differ. The hub provides a unified address map for these task-dependent parameters, and task switching is implemented as a bulk MMIO write of the parameter blob, achieving sub-1 ms switching latency without any FPGA fabric reconfiguration. The data path is not perturbed; the switching infrastructure is orthogonal to the streaming dataflow.

4.6 Pipeline Optimization of the MVAU + Adapter Integrated System

The integrated MVAU + Adapter system proposed in this thesis is explicitly pipeline-optimised. Barkaïc partitions the XNOR, popcount, Q8 contribution scaling, and accumulation operations of the integrated datapath into separate pipeline stages, inserts pipeline registers across the SIMD PE lanes of the Super Wrapper, and selects per-layer PE/SIMD unfolding factors that match the pipeline depth. The resulting integrated design runs at a higher achievable clock frequency and with a higher throughput than its unpipelined counterpart.

Table 4.2 compares the per-layer PE/SIMD configuration of the pipelined Barkaïc system against an unpipelined version of the same integrated datapath. The pipelined Barkaïc system selects substantially higher PE factors for Conv1–Conv5 ($1 \rightarrow 4\text{--}32$), unfolding the integrated MVAU + Adapter datapath to sustain the streaming throughput. Fully-connected layers FC1–FC3, whose datapaths do not host an adapter, retain their original configurations.

Table 4.2: PE/SIMD configuration per layer before and after the pipeline-aware redesign of the MVAU + Adapter integration.

Layer	Pipeline		No Pipeline	
	PE	SIMD	PE	SIMD
Conv1	16	3	1	3
Conv2	32	32	1	32
Conv3	16	32	1	32
Conv4	16	32	1	32
Conv5	4	32	1	32
Conv6	1	32	1	32
FC1	1	4	1	4
FC2	1	8	1	8
FC3	5	1	5	1

4.7 End-to-End Design Flow

The complete Barkaic design flow spans six stages from software training to hardware verification:

1. **Stage 1 — Backbone Pre-training:** The CNV backbone (6 conv + 3 FC layers) is trained on CIFAR-10 using PyTorch + Brevitas with 1W1A quantization-aware training. Weights are quantized to $\{-1, +1\}$ (1-bit signed), activations to $\{-1, +1\}$ (1-bit signed), and input to 8-bit (Q1.7 format) [44, 46, 47].
2. **Stage 2 — Adapter Fine-tuning:** The backbone weights are frozen. Only the adapter parameters (down/up projection weights, RC biases, and scaling factors α) are trained on the target dataset (SVHN or FashionMNIST).
3. **Stage 3 — Weight Export:** Backbone weights are exported in FINN’s interleaved format. Adapter weights are converted to hexadecimal `.dat` files for ROM initialization, with PE-aligned packing for the up-projection weights.
4. **Stage 4 — FINN Backbone Generation:** The FINN framework converts the quantized backbone into a streaming dataflow accelerator, generating HLS-based MVAU IP cores

with AXI-Stream interfaces.

5. **Stage 5 — RTL Integration:** The FINN-generated MVAU modules are replaced with Super Wrappers in Vivado. The original MVAU’s internal threshold logic is decoupled, and the custom Adapter IP, Stream Splitter, FIFO, and Stream Adder + Threshold modules are connected.
6. **Stage 6 — FPGA Verification:** The integrated design is synthesized, placed, and routed using Vivado 2022.2. A golden model is established in Python/Brevitas, and the FPGA output is compared bit-by-bit against the golden reference for 10,000 test samples to ensure 100% bit-exact match.

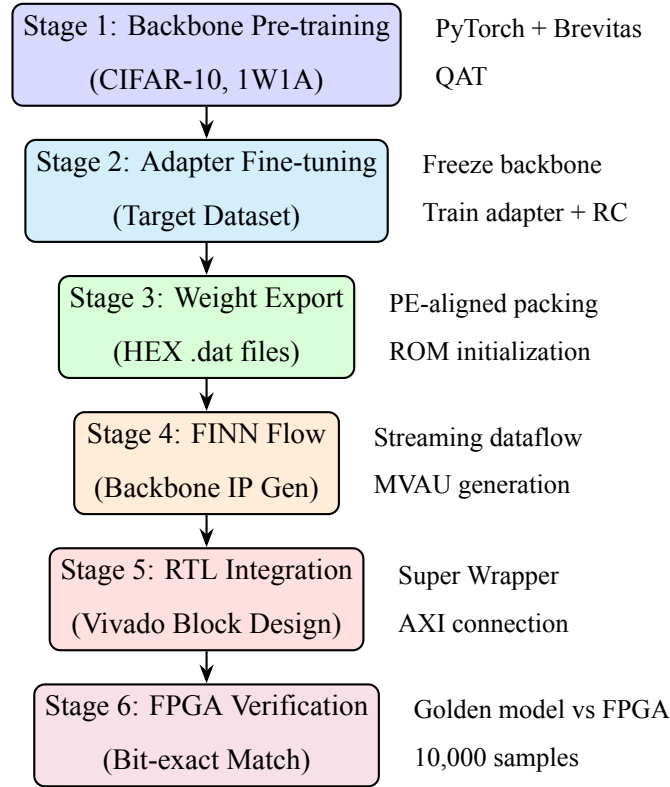


Figure 4.5: End-to-end software-to-hardware design flow for the Barkaic accelerator.

4.8 Verification Methodology

The verification strategy employs a three-level hierarchical approach covering per-module RTL simulation, top-level streaming dataflow simulation, and end-to-end on-board validation:

1. **Module-level RTL simulation:** Each Super Wrapper (MVAU1–MVAU5) is tested independently with golden test vectors generated by Python scripts that replicate the

exact hardware-equivalent computation (XNOR-popcount, accumulation, Q8 threshold comparison, sign/polarity handling). Critically, the golden generator reads the FINN-emitted `memblock.dat` binary convolution weights directly, because adapter-aware training modifies the backbone weights and therefore the original Brevitas checkpoint no longer matches what FINN compiled into `memblock`. Using `F.conv2d` over the Brevitas weights would introduce a 1–2% bit-mismatch; reading `memblock` eliminates this class of false failures.

2. **Golden data generation:** For each layer, the Python generator loads the `memblock`-derived MVAU weights, the pre-packed adapter down/up ROMs, the Q8 threshold ROM, and the RC biases, then computes outputs using exact integer arithmetic matching the RTL behaviour. The resulting `.dat` files (`mvau{N}_in.dat` / `mvau{N}_expected.dat`) serve as reference vectors for Xsim. The RC bias rounding discrepancy between PyTorch (`floor(bias/2 - IN_CH/2)`) and the exported `.dat` files (off by 1 in pathological cases) is sidestepped by reading the `.dat` file directly in the golden generator.
3. **Top-level dataflow simulation:** The complete streaming pipeline—Input DMA, Conv1 MVAU, five Adapter-enhanced Super Wrappers, MaxPool, FC1–FC3 MVAUs, LabelSelect, Output DMA—is simulated as a single Vivado block design with `tb_top.v` driving a batch of SVHN test images through the full dataflow. Classification decisions match the PyTorch golden reference, confirming that the adapter integration, FIFO synchronisation, and Stream Adder + Threshold fusion all behave identically to software at the block-design level. The on-board validation run below reconfirms this behaviour at scale.
4. **End-to-end on-board verification:** The bitstream is loaded onto PYNQ-Z2 and the SVHN test set is processed end-to-end. The on-board accuracy matches the PyTorch golden reference to within a small percentage-point gap consistent with the top-level simulation, confirming that the full streaming pipeline behaves identically to the software model.

The per-module simulation results are:

Table 4.3: Module-level RTL simulation results. Test vector counts correspond to per-channel outputs emitted over 10 sample inputs at each layer’s spatial output footprint.

Module	Test Vectors	Result
MVAU1 (Conv2)	15,680	100% Pass
MVAU2 (Conv3)	11,520	100% Pass
MVAU3 (Conv4)	8,000	100% Pass
MVAU4 (Conv5)	5,760	100% Pass
MVAU5 (Conv6)	2,560	100% Pass
Total	43,520	100% Pass

Bibliography

- [1] L. Gao, Z. Luo, and L. Wang, “Convolutional Neural Network Acceleration Techniques Based on FPGA Platforms: Principles, Methods, and Challenges,” *Information*, vol. 16, no. 10, art. 914, 2025. DOI: 10.3390/info16100914.
- [2] B. A. Arend, A. F. Ely, F. L. Kastensmidt, and M. Recamonde-Mendoza, “Predicting Hardware Resource Allocation of MVAU Modules in FINN-Generated FPGA Accelerators,” in *Proceedings of the Escola Regional de Alto Desempenho e Inteligência Artificial—Região Sul (ERAD-RS)*, Sociedade Brasileira de Computação, 2025. DOI: 10.5753/eramirs.2025.16671.
- [3] P. Antunes and A. Podobas, “FPGA-Based Neural Network Accelerators for Space Applications: A Survey,” *arXiv preprint arXiv:2504.16173*, 2025.
- [4] T. Lu and C. Chang, “Cache-Adapter: Efficient video action recognition using adapter fine-tuning and cache memorization technique,” *J. Visual Communication and Image Representation*, vol. 111, art. 104543, 2025. DOI: 10.1016/j.jvcir.2025.104543.
- [5] J. Jiang, Y. Zhou, Y. Gong, H. Yuan, and S. Liu, “FPGA-based Acceleration for Convolutional Neural Networks: A Comprehensive Review,” *arXiv preprint arXiv:2505.13461*, 2025.
- [6] R. Li, “Dataflow & Tiling Strategies in Edge-AI FPGA Accelerators: A Comprehensive Literature Review,” *arXiv preprint arXiv:2505.08992*, 2025.
- [7] L. Jungemann, B. Wintermann, H. Riebler, and C. Plessl, “FINN-HPC: Closing the Gap for Energy-Efficient Neural Network Inference on FPGAs in HPC,” in *Proceedings of the HEART*, ACM, 2025. DOI: 10.1145/3728179.3728189.

- [8] M. Tasci, A. Istanbulu, V. Tumen, and S. Kosunalp, “FPGA-QNN: Quantized Neural Network Hardware Acceleration on FPGAs,” *Applied Sciences*, vol. 15, no. 2, art. 688, 2025. DOI: 10.3390/app15020688.
- [9] H. Kwak et al., “LoRA-Edge: Tensor-Train-Assisted LoRA for Practical CNN Fine-Tuning on Edge Devices,” *arXiv preprint arXiv:2511.03765*, 2025.
- [10] R. Belanec et al., “PEFT-Bench: A Parameter-Efficient Fine-Tuning Methods Benchmark,” *arXiv preprint arXiv:2511.21285*, 2025.
- [11] M. E. Fouda, A. M. Eltawil, et al., “Quantized Convolutional Neural Networks: A Hardware Perspective,” *Frontiers in Electronics*, vol. 6, 2025. DOI: 10.3389/f-elec.2025.1469802.
- [12] A. El Bouazzaoui, O. Mouhib, and A. Hadjoudja, “NeuroAdaptiveNet: A Reconfigurable FPGA-Based Neural Network System with Dynamic Model Selection,” *Chips (MDPI)*, vol. 4, no. 2, art. 24, 2025. DOI: 10.3390/chips4020024.
- [13] A. Boudjadar, S. Islam, and R. Buyya, “Dynamic FPGA Reconfiguration for Scalable Embedded AI: A Co-Design Methodology for CNN Acceleration,” *Future Generation Computer Systems*, vol. 169, art. 107777, 2025. DOI: 10.1016/j.future.2025.107777.
- [14] M. Kang and D. Park, “Runtime-Robust Edge Inference System with Masking-Based Partial Update on Dynamic Reconfigurable FPGA,” *Sensors*, vol. 25, no. 24, art. 7448, 2025. DOI: 10.3390/s25247448.
- [15] A. M. Z. Khaki and A. Choi, “Optimizing Deep Learning Acceleration on FPGA for Real-Time and Resource-Efficient Image Classification,” *Applied Sciences*, vol. 15, no. 1, art. 422, 2025. DOI: 10.3390/app15010422.
- [16] J.-F. Schulte et al., “hls4ml: A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware,” *ACM Trans. Reconfigurable Technology and Systems*, 2025. DOI: 10.1145/3801979.
- [17] B. X. B. Yu et al., “Visual Tuning,” *ACM Computing Surveys*, vol. 56, no. 12, art. 297, 2024. DOI: 10.1145/3657632.

- [18] A. Rangsikunpum, S. Amiri, and L. Ost, “A Reconfigurable Coarse-to-Fine Approach for the Execution of CNN Inference Models in Low-Power Edge Devices,” *IET Computers & Digital Techniques*, 2024. DOI: 10.1049/cdt2/6214436.
- [19] F. Manca, F. Ratto, and F. Palumbo, “ONNX-to-Hardware Design Flow for Adaptive Neural-Network Inference on FPGAs,” *arXiv preprint arXiv:2406.09078*, 2024.
- [20] C.-H. Huang, S.-W. Tang, and P.-A. Hsiung, “ACNNE: An Adaptive Convolution Engine for CNNs Acceleration Exploiting Partial Reconfiguration on FPGAs,” in *Proceedings of the IEEE ISCAS*, 2024. DOI: 10.1109/ISCAS58744.2024.10558457.
- [21] H. Chen, R. Tao, H. Zhang, et al., “Conv-Adapter: Exploring Parameter Efficient Transfer Learning for ConvNets,” in *Proceedings of the IEEE/CVF CVPR Workshops*, 2024, pp. 1551–1561.
- [22] C. Ding, X. Cao, J. Xie, et al., “LoRA-C: Parameter-Efficient Fine-Tuning of Robust CNN for IoT Devices,” *arXiv preprint arXiv:2410.16954*, 2024.
- [23] H.-I. Liu et al., “Lightweight Deep Learning for Resource-Constrained Environments: A Survey,” *ACM Computing Surveys*, vol. 56, no. 10, art. 267, 2024. DOI: 10.1145/3657282.
- [24] Z. Zhao, Y. Chen, P. Feng, et al., “A High-Throughput FPGA Accelerator for Lightweight CNNs With Balanced Dataflow,” *arXiv preprint arXiv:2407.19449*, 2024.
- [25] A. El Bouazzaoui, A. Hadjoudja, O. Mouhib, and N. Cherkaoui, “FPGA-based ML Adaptive Accelerator: A Partial Reconfiguration Approach for Optimized ML Accelerator Utilization,” *Array*, vol. 21, art. 100337, 2024. DOI: 10.1016/j.array.2024.100337.
- [26] F. Minnella, T. Urso, M. T. Lazarescu, and L. Lavagno, “Design and Optimization of Residual Neural Network Accelerators for Low-Power FPGAs Using High-Level Synthesis,” *arXiv preprint arXiv:2309.15631*, 2023.
- [27] A. Montgomerie-Corcoran, P. Toupas, Z. Yu, and C.-S. Bouganis, “SATAY: A Streaming Architecture Toolflow for Accelerating YOLO Models on FPGA Devices,” *arXiv preprint arXiv:2309.01587*, 2023.
- [28] V. C. Johnson, J. S. Bali, S. Tanvashi, and C. B. Kolanur, “FPGA-Based Hardware Acceleration Using PYNQ-Z2,” in *Proceedings of the ICEEICT*, 2023. DOI: 10.1109/ICEE-ICT56924.2023.10157764.

- [29] S. Jie, H. Wang, and Z.-H. Deng, “Revisiting the Parameter Efficiency of Adapters from the Perspective of Precision Redundancy,” in *Proceedings of the IEEE/CVF ICCV*, 2023. arXiv:2307.16867.
- [30] S. A. Alam, D. Gregg, G. Gambardella, T. Preusser, and M. Blott, “On the RTL Implementation of FINN Matrix Vector Unit,” *ACM Trans. Embedded Computing Systems*, vol. 22, no. 6, 2023. DOI: 10.1145/3547141.
- [31] A. Nechi, L. Groth, S. Mulhem, et al., “FPGA-based Deep Learning Inference Accelerators: Where Are We Standing?” *ACM Trans. Reconfigurable Technology and Systems*, vol. 16, no. 4, art. 60, 2023. DOI: 10.1145/3613963.
- [32] Y. Su, K. P. Seng, L. M. Ang, and J. Smith, “Binary Neural Networks in FPGAs: Architectures, Tool Flows and Hardware Comparisons,” *Sensors*, vol. 23, no. 22, art. 9254, 2023. DOI: 10.3390/s23229254.
- [33] J.-Y. Zhan, A.-T. Yu, W. Jiang, et al., “FPGA-based Acceleration for Binary Neural Networks in Edge Computing,” *J. Electronic Science and Technology*, vol. 21, no. 2, art. 100204, 2023. DOI: 10.1016/j.jnlest.2023.100204.
- [34] H.-J. Kang and B.-D. Yang, “AoCStream: All-on-Chip CNN Accelerator with Stream-Based Line-Buffer Architecture and Accelerator-Aware Pruning,” *Sensors*, vol. 23, no. 19, art. 8104, 2023. DOI: 10.3390/s23198104.
- [35] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proceedings of the ICLR*, 2022. arXiv:2106.09685.
- [36] M. Nagel, M. Fournarakis, R. A. Amjad, et al., “A White Paper on Neural Network Quantization,” *arXiv preprint arXiv:2106.08295*, 2021.
- [37] Z. Liu, Z. Shen, M. Savvides, and K.-T. Cheng, “ReActNet: Towards Precise Binary Neural Network with Generalized Activation Functions,” in *Proceedings of the ECCV*, 2020.
- [38] H. Qin, R. Gong, X. Liu, et al., “Binary Neural Networks: A Survey,” *Pattern Recognition*, vol. 105, art. 107281, 2020. DOI: 10.1016/j.patcog.2020.107281.
- [39] P. N. Whatmough et al., “FixyNN: Efficient Hardware for Mobile Computer Vision via Transfer Learning,” in *Proceedings of the MLSys*, 2019.

- [40] N. Houlsby, A. Giurciu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *Proceedings of the ICML*, 2019, pp. 2790–2799.
- [41] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, “FP-BNN: Binarized Neural Network on FPGA,” *Neurocomputing*, vol. 275, pp. 1072–1086, 2018.
- [42] Z. Liu, B. Wu, W. Luo, et al., “Bi-Real Net: Enhancing the Performance of 1-bit CNNs With Improved Representational Capability and Advanced Training Algorithm,” in *Proceedings of the ECCV*, 2018.
- [43] M. Blott, T. B. Preußer, N. J. Fraser, et al., “FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks,” *ACM Trans. Reconfigurable Technology and Systems*, vol. 11, no. 3, 2018. DOI: 10.1145/3242897.
- [44] B. Jacob, S. Kligys, B. Chen, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE/CVF CVPR*, 2018, pp. 2704–2713.
- [45] A. Pappalardo, “Xilinx/brevitas,” *Zenodo*, 2021. DOI: 10.5281/zenodo.3333552.
- [46] Y. Umuroglu, N. J. Fraser, G. Gambardella, et al., “FINN: A Framework for Fast, Scalable Binarized Neural Network Inference,” in *Proceedings of the FPGA*, ACM, 2017, pp. 65–74.
- [47] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, “XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks,” in *Proceedings of the ECCV*, 2016, pp. 525–542.
- [48] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE CVPR*, 2016, pp. 770–778.
- [49] M. Courbariaux, Y. Bengio, and J.-P. David, “BinaryConnect: Training Deep Neural Networks with binary weights during propagations,” in *Proceedings of the NeurIPS*, 2015, pp. 3123–3131.
- [50] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, “Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations,” *J. Machine Learning Research*, vol. 18, no. 187, pp. 1–30, 2018. (Originally arXiv:1609.07061, 2016.)

- [51] A. Pappalardo, “Brevitas: A Quantization-Aware Training Library,” *GitHub Repository*, Xilinx Research Labs, 2021. <https://github.com/Xilinx/brevitas>.